

# Community Sports Facilities Booking System

Design, Implementation and Testing of a Web-Based Database Application

Module code and name: \_\_\_\_\_

Student name: \_\_\_\_\_

Student ID: \_\_\_\_\_

Module leader: \_\_\_\_\_

Submission date: \_\_\_\_\_

Assignment No: 001

AI assistance declaration

OpenAI Codex assisted with code changes, SQL checks, automated testing, screenshots, report drafting and formatting. The student is responsible for reviewing the work, understanding the implementation and following the institution's rules on permitted AI use.

Student signature: \_\_\_\_\_ Date: \_\_\_\_\_

# Table of Contents

|                                                       |    |
|-------------------------------------------------------|----|
| Table of Contents.....                                | 2  |
| 1. Introduction.....                                  | 4  |
| 1.1 Scope and assumptions.....                        | 4  |
| 2. Specification of Database Relations .....          | 5  |
| 2.1 Relational Entity-Relationship Diagram.....       | 5  |
| 2.1.1 Relationships and cardinality .....             | 5  |
| 2.1.2 Normalisation.....                              | 6  |
| 2.2 Data Dictionary.....                              | 6  |
| 2.2.1 Member .....                                    | 6  |
| 2.2.2 FacilityType .....                              | 6  |
| 2.2.3 Facility.....                                   | 7  |
| 2.2.4 Booking.....                                    | 7  |
| 2.2.5 Review.....                                     | 7  |
| 2.2.6 SportPreference .....                           | 7  |
| 2.2.7 Inquiry .....                                   | 8  |
| 2.3 SQL Developer Data Modeler .....                  | 8  |
| 3. Database Implementation.....                       | 9  |
| 3.1 Data Models.....                                  | 9  |
| 3.2 Insert Statements.....                            | 9  |
| 3.3 Select Queries .....                              | 10 |
| 3.3.1 Bookings with member and facility names .....   | 11 |
| 3.3.2 Facilities available for a selected time .....  | 12 |
| 3.3.3 Average rating for each reviewed facility ..... | 13 |
| 3.3.4 Five least expensive facilities.....            | 14 |
| 3.3.5 Members who have made a booking.....            | 15 |
| 3.3.6 Facilities with no booking records .....        | 16 |
| 3.3.7 Preferred sports for each member .....          | 17 |
| 3.3.8 Latest guest inquiries.....                     | 18 |
| 4. Implementation of the Web-Based Application.....   | 19 |
| 4.1 System Architecture .....                         | 19 |
| 4.2 Home Page.....                                    | 19 |
| 4.3 Member Functionality .....                        | 20 |
| 4.3.1 Sign In .....                                   | 20 |
| 4.3.2 Registration .....                              | 21 |
| 4.3.3 Facility Search.....                            | 24 |
| 4.3.4 Facility Booking.....                           | 25 |

|                                                   |    |
|---------------------------------------------------|----|
| 4.3.5 Review Submission .....                     | 28 |
| 4.4 Guest Functionality.....                      | 29 |
| 4.4.1 Restricted Search.....                      | 29 |
| 4.4.2 Review Search.....                          | 30 |
| 4.4.3 Membership Registration .....               | 31 |
| 4.4.4 Send Inquiry .....                          | 31 |
| 5. Testing .....                                  | 34 |
| 5.1 Test Evidence .....                           | 35 |
| 5.2 Summary of Testing .....                      | 36 |
| 6. Further Discussion and Reflection.....         | 38 |
| 7. Conclusion .....                               | 38 |
| 8. References.....                                | 38 |
| 9. Appendices.....                                | 40 |
| Appendix A Full database schema .....             | 40 |
| Member .....                                      | 40 |
| FacilityType .....                                | 40 |
| Facility.....                                     | 40 |
| Booking.....                                      | 40 |
| Review.....                                       | 41 |
| SportPreference .....                             | 41 |
| Inquiry .....                                     | 41 |
| Appendix B Sample insert script .....             | 41 |
| Appendix C Running and verifying the project..... | 42 |
| Appendix D Full controller code .....             | 42 |

# **1. Introduction**

The Community Sports Facilities Booking System is a web application for a municipal sports council. It helps members find local venues and request a suitable time slot. The application checks existing bookings before saving a new request, so overlapping bookings can be rejected.

Registered members can sign in, record preferred sports, search facilities by sport, location, date and time, request bookings and review facilities after a confirmed visit. Guests can browse general availability, read reviews, register and send inquiries. Guests cannot create bookings or submit reviews.

The prototype uses Microsoft SQL Server, ASP.NET Core MVC with C#, Entity Framework Core and Razor views. The database contains Member, FacilityType, Facility, Booking, Review, SportPreference and Inquiry. SQL Developer Data Modeler was used for the supplied design. The report explains the database, implementation, SQL queries, testing and remaining limitations.

## **1.1 Scope and assumptions**

A facility supports one booking for an overlapping time slot. Adjacent bookings are allowed when one ends exactly as another starts.

New bookings have Pending status. Confirmation is a manual database task because an administrative approval page is outside the prototype.

Reviews require a Confirmed booking whose end time is in the past. More than one review is currently allowed.

All dates and times use the server's local time. Searches containing only a date show facilities with no active booking that day.

The prototype has no payment processing, email confirmation or password recovery. Passwords remain plain text and must be hashed before production use.

## 2. Specification of Database Relations

### 2.1 Relational Entity-Relationship Diagram

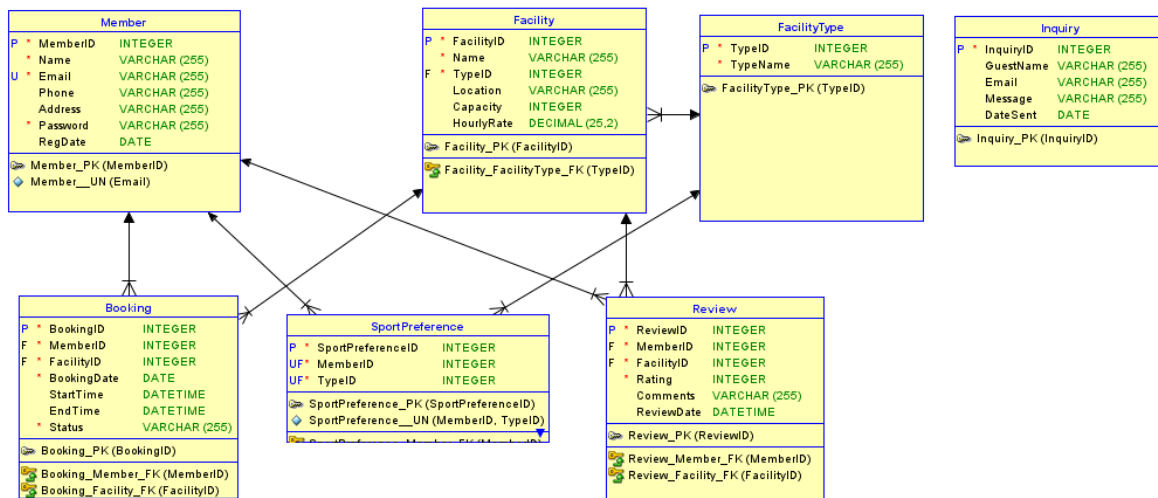


Figure 2.1. Relational model exported from SQL Developer Data Modeler.

The relational model contains seven entities. The diagram is consistent with the implemented relationships, including Booking to Member and Booking to Facility. SportPreference has its own primary key and a unique MemberID–TypeID pair. Inquiry is independent because a guest does not need a Member record.

**Member.** Stores each registered person’s name, email, contact details, password and registration date. Email is unique and is used for login.

**FacilityType.** Stores sport categories. Facilities and member preferences refer to a TypeID instead of repeating category names.

**Facility.** Stores venue name, sport category, location, capacity and hourly rate.

**Booking.** Links one member to one facility and stores the requested date, start time, end time and booking status.

**Review.** Links the author and facility and stores an integer rating from 1 to 5, comments and a review date.

**SportPreference.** Resolves the many-to-many relationship between members and sport categories. A member can choose several sports, but cannot select the same category twice.

**Inquiry.** Stores a guest’s name, email, message and date. It has no foreign key to Member.

#### 2.1.1 Relationships and cardinality

| Parent       | Child           | Cardinality |
|--------------|-----------------|-------------|
| Facility     | Review          | One to many |
| Facility     | Booking         | One to many |
| FacilityType | Facility        | One to many |
| FacilityType | SportPreference | One to many |
| Member       | Review          | One to many |

| Parent | Child           | Cardinality |
|--------|-----------------|-------------|
| Member | SportPreference | One to many |
| Member | Booking         | One to many |

Every child foreign key is required. One parent can have no related child records or many of them; every child row refers to exactly one parent. SportPreference turns the many-to-many association into two one-to-many relationships.

### 2.1.2 Normalisation

The design separates different subjects into their own tables. In first normal form, each column holds a single value, and preferred sports are represented by separate SportPreference rows. The non-key attributes describe their table's key rather than only part of a composite candidate key, supporting second normal form. FacilityType holds the sport name once, while Facility and SportPreference hold its ID, avoiding a repeated type name and supporting third normal form.

## 2.2 Data Dictionary

The following tables describe the inspected SQL Server schema after reconciliation. Existing VARCHAR(255) lengths and DECIMAL(25,2) rates are retained to avoid truncating saved data. Earlier narrower lengths in the draft did not describe the actual database. Blank optional values are permitted only where NULL is shown.

### 2.2.1 Member

| Attribute | SQL type     | Key and constraints    | Description              |
|-----------|--------------|------------------------|--------------------------|
| MemberID  | INT          | NOT NULL; IDENTITY; PK | Member identifier        |
| Name      | VARCHAR(255) | NOT NULL               | Name                     |
| Email     | VARCHAR(255) | NOT NULL; UNIQUE       | Email address            |
| Phone     | VARCHAR(255) | NULL allowed           | Contact number           |
| Address   | VARCHAR(255) | NULL allowed           | Postal address           |
| Password  | VARCHAR(255) | NOT NULL               | Prototype login password |
| RegDate   | DATE         | NULL allowed           | Registration date        |

### 2.2.2 FacilityType

| Attribute | SQL type     | Key and constraints    | Description               |
|-----------|--------------|------------------------|---------------------------|
| TypeID    | INT          | NOT NULL; IDENTITY; PK | Sport category identifier |
| TypeName  | VARCHAR(255) | NOT NULL               | Sport category            |

### 2.2.3 Facility

| Attribute  | SQL type      | Key and constraints          | Description               |
|------------|---------------|------------------------------|---------------------------|
| FacilityID | INT           | NOT NULL; IDENTITY; PK       | Facility identifier       |
| Name       | VARCHAR(255)  | NOT NULL                     | Name                      |
| TypeID     | INT           | NOT NULL; FK to FacilityType | Sport category identifier |
| Location   | VARCHAR(255)  | NULL allowed                 | Venue location            |
| Capacity   | INT           | NULL allowed                 | Maximum participants      |
| HourlyRate | DECIMAL(25,2) | NULL allowed                 | Rate in LKR per hour      |

### 2.2.4 Booking

| Attribute   | SQL type     | Key and constraints              | Description                     |
|-------------|--------------|----------------------------------|---------------------------------|
| BookingID   | INT          | NOT NULL; IDENTITY; PK           | Booking identifier              |
| MemberID    | INT          | NOT NULL; FK to Member           | Member identifier               |
| FacilityID  | INT          | NOT NULL; FK to Facility         | Facility identifier             |
| BookingDate | DATE         | NOT NULL                         | Requested date                  |
| StartTime   | DATETIME     | NULL allowed                     | Start date and time             |
| EndTime     | DATETIME     | NULL allowed                     | End date and time               |
| Status      | VARCHAR(255) | NOT NULL; CHECK allowed statuses | Pending, Confirmed or Cancelled |

### 2.2.5 Review

| Attribute  | SQL type     | Key and constraints      | Description              |
|------------|--------------|--------------------------|--------------------------|
| ReviewID   | INT          | NOT NULL; IDENTITY; PK   | Review identifier        |
| MemberID   | INT          | NOT NULL; FK to Member   | Member identifier        |
| FacilityID | INT          | NOT NULL; FK to Facility | Facility identifier      |
| Rating     | INT          | NOT NULL; CHECK 1–5      | Whole number from 1 to 5 |
| Comments   | VARCHAR(255) | NULL allowed             | Written feedback         |
| ReviewDate | DATETIME     | NULL allowed             | Feedback date and time   |

### 2.2.6 SportPreference

| Attribute         | SQL type | Key and constraints                             | Description           |
|-------------------|----------|-------------------------------------------------|-----------------------|
| SportPreferenceID | INT      | NOT NULL; IDENTITY; PK                          | Preference identifier |
| MemberID          | INT      | NOT NULL; Unique pair with TypeID; FK to Member | Member identifier     |

| Attribute | SQL type | Key and constraints                                     | Description               |
|-----------|----------|---------------------------------------------------------|---------------------------|
| TypeID    | INT      | NOT NULL; Unique pair with MemberID; FK to FacilityType | Sport category identifier |

### 2.2.7 Inquiry

| Attribute | SQL type     | Key and constraints    | Description        |
|-----------|--------------|------------------------|--------------------|
| InquiryID | INT          | NOT NULL; IDENTITY; PK | Inquiry identifier |
| GuestName | VARCHAR(255) | NULL allowed           | Guest sender       |
| Email     | VARCHAR(255) | NULL allowed           | Email address      |
| Message   | VARCHAR(255) | NULL allowed           | Inquiry text       |
| DateSent  | DATE         | NULL allowed           | Inquiry date       |

## 2.3 SQL Developer Data Modeler

The supplied design is saved as ER.dmd with its ER model folder. The original logical diagram was engineered into a relational model. The relational export in Figure 2.1 shows the final seven-table design and foreign keys. The saved DDL identifies SQL Developer Data Modeler 24.3.1.351.0831 and a SQL Server 2012 generation target. This is a DDL target setting, not a claim about the installed SQL Server version.

```
-- Generated by Oracle SQL Developer Data Modeler 24.3.1.351.0831
-- at:      2026-08-27 18:17:18 IST
-- site:    SQL Server 2012
-- type:    SQL Server 2012
```

The generated script was checked against SQL Server metadata. The supplied DDL text omitted Booking\_Member\_FK even though the relational diagram showed it; the running database contains that foreign key. Review.Rating was still VARCHAR in the running database. It was converted to INT after checking every saved value, and database CHECK constraints were added for Rating and Status. Appendix A therefore records the reconciled database rather than repeating an outdated generated script.

The original logical export is an earlier draft: it lacks SportPreferenceID and has relationship lines that do not fully match the physical schema. Figure 2.1 and the relationship table above define the implemented design. The saved project, original exports and reconciled DDL are retained as design evidence.

## 3. Database Implementation

### 3.1 Data Models

SQL Server stores all seven tables. Primary keys use IDENTITY(1,1), foreign keys preserve references, Member.Email is unique and the SportPreference pair is unique. EF Core maps BookingID as database-generated and MemberID as a supplied foreign key. The previous mapping reversed these settings; the corrected mapping allows a new booking to receive its own identifier.

```
entity.Property(e => e.BookingId).ValueGeneratedOnAdd();  
entity.Property(e => e.MemberId).ValueGeneratedNever();
```

Tables can be created in dependency order: Member and FacilityType, then Facility, followed by Booking, Review and SportPreference. Inquiry has no dependencies. Appendix A contains the complete executable CREATE TABLE script extracted from the inspected schema.

#### Database check constraints

SportsBooking · SQL Server result captured 2026-09-17

| TableName | ConstraintName    | Definition                                                            |
|-----------|-------------------|-----------------------------------------------------------------------|
| Review    | CK_Review_Rating  | (([Rating]>=(1) AND [Rating]<=(5))                                    |
| Booking   | CK_Booking_Status | (([Status]='Cancelled' OR [Status]='Confirmed' OR [Status]='Pending') |

Figure 3.1. Actual SQL Server CHECK constraint metadata, displayed as a result table.

### 3.2 Insert Statements

The database already contains representative sample records. The captured counts below exclude the temporary records created for testing and screenshots. Existing data includes duplicate facility names with different IDs, so relationships and queries use IDs rather than assuming names are unique.

| Table        | Saved rows |
|--------------|------------|
| Member       | 8          |
| FacilityType | 5          |
| Facility     | 18         |

| Table           | Saved rows |
|-----------------|------------|
| Booking         | 15         |
| Review          | 20         |
| SportPreference | 15         |
| Inquiry         | 8          |

Appendix B provides a standalone example that inserts into every table in dependency order. SCOPE\_IDENTITY() captures each new key instead of assuming IDs begin at 1. The example is intended to be run once in a test database; it was not added to the existing database. Successful application inserts are demonstrated by the captured Member, SportPreference, Booking, Review and Inquiry rows in Section 4.

```
INSERT INTO Facility (Name, TypeID, Location, Capacity, HourlyRate)
VALUES ('Sample Tennis Court', @TypeID, 'Negombo', 4, 1500.00);
DECLARE @FacilityID INT = SCOPE_IDENTITY();
```

### 3.3 Select Queries

Eight queries were executed against SportsBooking. Each result image displays the actual returned rows in a readable table; these are captured query outputs, not screenshots of SQL Server Management Studio. No passwords are selected. The availability example uses 17 October 2026, 10:00–11:00.

### 3.3.1 Bookings with member and facility names

This three-table join replaces raw foreign keys with readable member and facility names for a booking list.

```
SELECT TOP (10) b.BookingID,m.Name AS Member,f.Name AS Facility,b.BookingDate,b.Status
FROM Booking b
JOIN Member m ON b.MemberID=m.MemberID
JOIN Facility f ON b.FacilityID=f.FacilityID
ORDER BY b.BookingID;
```

#### Bookings with member and facility names

SportsBooking · SQL Server result captured 2026-09-17

| BookingID | Member         | Facility                | BookingDate | Status    |
|-----------|----------------|-------------------------|-------------|-----------|
| 1         | M A M SHAN     | Central Tennis Court 1  | 2025-03-10  | Confirmed |
| 2         | M A M SHAN     | Indoor Basketball Court | 2025-03-14  | Confirmed |
| 3         | Amal Perera    | Badminton Hall A        | 2025-03-11  | Confirmed |
| 4         | Amal Perera    | Municipal Swimming Pool | 2025-03-18  | Pending   |
| 5         | Nisha Fernando | Riverside Soccer Field  | 2025-03-12  | Confirmed |
| 6         | Nisha Fernando | Central Tennis Court 1  | 2025-03-20  | Cancelled |
| 7         | Nisha Fernando | Indoor Basketball Court | 2025-03-22  | Pending   |
| 8         | Ruwan Silva    | Badminton Hall A        | 2025-03-15  | Confirmed |
| 9         | Ruwan Silva    | Badminton Hall B        | 2025-03-19  | Confirmed |
| 10        | Ruwan Silva    | Municipal Swimming Pool | 2025-03-25  | Pending   |

*Figure 3.2. Query 1 returned 10 rows.*

### 3.3.2 Facilities available for a selected time

The NOT EXISTS condition excludes overlapping Pending or Confirmed bookings. Cancelled bookings do not block a slot. Missing times are treated as blocking the requested day.

```
SELECT f.FacilityID,f.Name,f.Location
FROM Facility f
WHERE NOT EXISTS (SELECT 1
FROM Booking b
WHERE b.FacilityID=f.FacilityID AND b.BookingDate='2026-10-17' AND b.Status<>'Cancelled' AND
(b.StartTime IS NULL OR b.EndTime IS NULL OR (b.StartTime<'2026-10-17T11:00:00' AND
b.EndTime>'2026-10-17T10:00:00'))
ORDER BY f.FacilityID;
```

#### Facilities available for a selected time

SportsBooking - SQL Server result captured 2026-09-17

| FacilityID | Name                    | Location                          |
|------------|-------------------------|-----------------------------------|
| 1          | Central Tennis Court 1  | Community Sports Complex, Negombo |
| 2          | Central Tennis Court 2  | Community Sports Complex, Negombo |
| 3          | Riverside Soccer Field  | Riverside Park, Gampaha           |
| 4          | Municipal Soccer Ground | Municipal Grounds, Colombo        |
| 5          | Indoor Basketball Court | Community Sports Complex, Negombo |
| 6          | Open Basketball Court   | Riverside Park, Gampaha           |
| 7          | Badminton Hall A        | Town Hall Sports Centre, Colombo  |
| 8          | Badminton Hall B        | Town Hall Sports Centre, Colombo  |
| 9          | Municipal Swimming Pool | Municipal Grounds, Colombo        |
| 10         | Central Tennis Court 1  | Community Sports Complex, Negombo |
| 11         | Central Tennis Court 2  | Community Sports Complex, Negombo |
| 12         | Riverside Soccer Field  | Riverside Park, Gampaha           |
| 13         | Municipal Soccer Ground | Municipal Grounds, Colombo        |
| 14         | Indoor Basketball Court | Community Sports Complex, Negombo |
| 15         | Open Basketball Court   | Riverside Park, Gampaha           |
| 16         | Badminton Hall A        | Town Hall Sports Centre, Colombo  |
| 17         | Badminton Hall B        | Town Hall Sports Centre, Colombo  |
| 18         | Municipal Swimming Pool | Municipal Grounds, Colombo        |

Figure 3.3. Query 2 returned 18 rows.

### 3.3.3 Average rating for each reviewed facility

GROUP BY produces one row per reviewed facility. Casting before AVG prevents integer division from hiding fractional averages.

```
SELECT f.Name,COUNT(*) AS Reviews,CAST(AVG(CAST(r.Rating AS DECIMAL(5,2)))) AS DECIMAL(5,2)) AS  
AverageRating  
FROM Review r  
JOIN Facility f ON r.FacilityID=f.FacilityID  
GROUP BY f.FacilityID,f.Name  
ORDER BY AverageRating DESC;
```

#### Average rating for each reviewed facility

SportsBooking · SQL Server result captured 2026-09-17

| Name                    | Reviews | AverageRating |
|-------------------------|---------|---------------|
| Badminton Hall A        | 4       | 4.5           |
| Indoor Basketball Court | 6       | 4.33          |
| Municipal Soccer Ground | 2       | 4             |
| Central Tennis Court 1  | 4       | 3             |
| Riverside Soccer Field  | 2       | 3             |
| Badminton Hall B        | 2       | 2             |

*Figure 3.4. Query 3 returned 6 rows.*

### 3.3.4 Five least expensive facilities

TOP and ORDER BY list five venues with the lowest known hourly rates. FacilityID breaks equal-price ties.

```
SELECT TOP (5) Name,Location,HourlyRate
FROM Facility
WHERE HourlyRate IS NOT NULL
ORDER BY HourlyRate,FacilityID;
```

#### Five least expensive facilities

SportsBooking · SQL Server result captured 2026-09-17

| Name                    | Location                         | HourlyRate |
|-------------------------|----------------------------------|------------|
| Municipal Swimming Pool | Municipal Grounds, Colombo       | 800        |
| Municipal Swimming Pool | Municipal Grounds, Colombo       | 800        |
| Badminton Hall A        | Town Hall Sports Centre, Colombo | 1200       |
| Badminton Hall B        | Town Hall Sports Centre, Colombo | 1200       |
| Badminton Hall A        | Town Hall Sports Centre, Colombo | 1200       |

*Figure 3.5. Query 4 returned 5 rows.*

### 3.3.5 Members who have made a booking

The subquery finds members with at least one booking, without duplicating a member who has several bookings.

```
SELECT MemberID,Name
FROM Member
WHERE MemberID IN (SELECT MemberID
FROM Booking)
ORDER BY MemberID;
```

#### Members who have made a booking

SportsBooking · SQL Server result captured 2026-09-17

| MemberID | Name              |
|----------|-------------------|
| 1        | M A M SHAN        |
| 2        | Amal Perera       |
| 3        | Nisha Fernando    |
| 4        | Ruwan Silva       |
| 5        | Dilani Jayasuriya |
| 9        | M A M SHAN        |

*Figure 3.6. Query 5 returned 6 rows.*

### 3.3.6 Facilities with no booking records

LEFT JOIN retains facilities even when there is no matching booking. Testing for a null booking ID identifies unused facilities.

```
SELECT f.FacilityID,f.Name
FROM Facility f
LEFT
JOIN Booking b ON f.FacilityID=b.FacilityID
WHERE b.BookingID IS NULL
ORDER BY f.FacilityID;
```

#### Facilities with no booking records

SportsBooking - SQL Server result captured 2026-09-17

| FacilityID | Name                    |
|------------|-------------------------|
| 6          | Open Basketball Court   |
| 10         | Central Tennis Court 1  |
| 11         | Central Tennis Court 2  |
| 12         | Riverside Soccer Field  |
| 13         | Municipal Soccer Ground |
| 14         | Indoor Basketball Court |
| 15         | Open Basketball Court   |
| 18         | Municipal Swimming Pool |

*Figure 3.7. Query 6 returned 8 rows.*

### 3.3.7 Preferred sports for each member

The link table joins members to their selected sport categories. This query demonstrates the many-to-many association.

```
SELECT m.Name AS Member,t.TypeName AS PreferredSport
FROM SportPreference s
JOIN Member m ON m.MemberID=s.MemberID
JOIN FacilityType t ON t.TypeID=s.TypeID
ORDER BY m.MemberID,t.TypeName;
```

#### Preferred sports for each member

SportsBooking · SQL Server result captured 2026-09-17

| Member            | PreferredSport |
|-------------------|----------------|
| M A M SHAN        | Basketball     |
| M A M SHAN        | Tennis         |
| Amal Perera       | Badminton      |
| Amal Perera       | Swimming       |
| Nisha Fernando    | Basketball     |
| Nisha Fernando    | Soccer         |
| Nisha Fernando    | Tennis         |
| Ruwan Silva       | Badminton      |
| Dilani Jayasuriya | Basketball     |
| Dilani Jayasuriya | Soccer         |
| M A M SHAN        | Soccer         |
| M A M SHAN        | Swimming       |
| M A M SHAN        | Tennis         |
| M A M SHAN        | Badminton      |
| M A M SHAN        | Basketball     |

Figure 3.8. Query 7 returned 15 rows.

### 3.3.8 Latest guest inquiries

The latest five inquiries are returned first, with InquiryID resolving ties in DateSent. This is an internal SQL report; the application does not expose inquiry records publicly.

```
SELECT TOP (5) InquiryID, GuestName, Message, DateSent
FROM Inquiry
ORDER BY DateSent DESC, InquiryID DESC;
```

#### Latest guest inquiries

SportsBooking · SQL Server result captured 2026-09-17

| InquiryID | GuestName          | Message                                                    | DateSent   |
|-----------|--------------------|------------------------------------------------------------|------------|
| 8         | Ishara Gunawardena | Can facilities be booked for a company sports day?         | 2025-03-17 |
| 4         | Ishara Gunawardena | Can facilities be booked for a company sports day?         | 2025-03-17 |
| 7         | Menaka Silva       | Is the swimming pool open on public holidays?              | 2025-03-13 |
| 3         | Menaka Silva       | Is the swimming pool open on public holidays?              | 2025-03-13 |
| 6         | Tharindu Rajapaksa | Do you run coaching sessions for children during holidays? | 2025-03-08 |

*Figure 3.9. Query 8 returned 5 rows.*

## 4. Implementation of the Web-Based Application

### 4.1 System Architecture

The project uses a simple MVC structure. Models describe the database entities and form inputs, controllers process requests, and Razor views render HTML. SportsContext is injected into controllers and uses the SQL Server provider to query and save data. EF Core reverse engineering produces entity classes and a DbContext from an existing database (Microsoft, n.d.-b).

Browser request

- > Controller action
- > Form view model and validation
- > SportsContext and EF Core
- > SQL Server
- > Razor view returned to the browser

| Area        | Files and purpose                                                                                                        |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| Controllers | Home, Account, Facility, Booking, Review and Inquiry handle public/member routes.                                        |
| Models      | Seven entity classes; RegisterViewModel, FacilitySearchViewModel, BookingViewModel and ReviewViewModel handle form data. |
| Data        | SportsContext contains table mappings and relationships.                                                                 |
| Views       | Razor pages share _Layout.cshtml and Bootstrap styling.                                                                  |
| Program.cs  | Registers MVC, SQL Server and sessions; serves static files and maps routes.                                             |
| tests       | SmokeTest.ps1 executes HTTP requests and checks saved database records.                                                  |

MVC separates request handling from page presentation (Microsoft, n.d.-a). Session stores MemberID and MemberName after a valid login. Protected actions check MemberID before accessing data. AddDistributedMemoryCache, AddSession and UseSession configure this prototype's in-memory sessions (Microsoft, n.d.-c). Restarting the app clears them.

```
builder.Services.AddDbContext<SportsContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("SportsDb")));

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
});
```

View models contain only editable fields. MemberId and booking Status are set by the server. Required fields, maximum lengths and numeric ranges are checked through data annotations and ModelState.IsValid (Microsoft, n.d.-d). POST forms include anti-forgery tokens.

### 4.2 Home Page

The home page introduces community sports, links to facility search and reviews, and presents separate member-login and new-registration areas. Navigation changes after login to include My Bookings and logout. Contact Us remains available to both members and guests.

## Community Sports Facilities

Find a local place to play tennis, football, basketball, badminton or go swimming.

The Sports Council helps the community stay active by providing facilities for practice, friendly games and group activities. Contact us to ask about sports programmes and membership.

[Browse facilities](#)[Read reviews](#)

### Already a member?

Sign in to check availability and request a booking.

[Member login](#)

### New member?

Register your details and choose your favourite sports.

[Register](#)

Have a question? [Contact the Sports Council](#).

Figure 4.1. Home page with login and registration options.

```
@if (Context.Session.GetInt32("MemberID") == null)
{
    <a asp-controller="Account" asp-action="Login">Member login</a>
    <a asp-controller="Account" asp-action="Register">Register</a>
}
```

## 4.3 Member Functionality

### 4.3.1 Sign In

AccountController checks that email and password have been supplied, looks up the member and displays an error when the details do not match. A valid login stores the member identity in session and redirects to Facility/Search. Logout is a POST action that clears the session.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Login(string email, string password)
{
    if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(password))
    {
        ModelState.AddModelError("", "Please enter your email and password.");
        return View();
    }

    var member = await _context.Members
        .FirstOrDefaultAsync(m => m.Email == email.Trim() && m.Password == password);

    if (member == null)
    {
        ModelState.AddModelError("", "Invalid email or password.");
        return View();
    }
}
```

```
HttpContext.Session.SetInt32("MemberID", member.MemberId);
HttpContext.Session.SetString("MemberName", member.Name);

return RedirectToAction("Search", "Facility");
```

SportsBookingSystem   Home   Facilities   Reviews   Contact Us   Register   Login

---

## Member Login

---

• Invalid email or password.

Email

Password

Login

Not a member yet? [Register here.](#)

---

© 2026 - SportsBookingSystem - [Privacy](#)

*Figure 4.2. Invalid credentials produce a validation message.*

The existing prototype compares plain-text passwords. This is a recorded limitation, not a production authentication design. Password hashing, account lockout and recovery are future improvements.

### 4.3.2 Registration

The registration form captures name, email, phone, address, password and at least one preferred sport. It rejects duplicate email addresses and sport IDs that do not exist. Repeated sport IDs are removed with `Distinct()`. The `Member` and its `SportPreferences` are saved in one `SaveChangesAsync` call, then the member is signed in.

## Become a Member

---

Name

Report Evidence Member

Email

Report1789668992879@example.test

Phone

0770000000

Address

Report Test Area

Password

\*\*\*\*\*

Preferred Sports

☐ Tennis

☐ Soccer

☐ Basketball

☐ Badminton

☐ Swimming

☒ Report1789668992879

Register

*Figure 4.3. Completed registration form with a preferred sport.*

```
foreach (var typeId in model.SelectedSports)
{
    member.SportPreferences.Add(new SportPreference { TypeId = typeId });
}
_context.Members.Add(member);
await _context.SaveChangesAsync();
```

### Saved member data

SportsBooking - SQL Server result captured 2026-09-17

| MemberID | Name                   | Email                            | RegDate    |
|----------|------------------------|----------------------------------|------------|
| 12       | Report Evidence Member | Report1789668992879@example.test | 2026-09-17 |

*Figure 4.4. Saved test member row, with the password excluded.*

### Saved preference data

SportsBooking - SQL Server result captured 2026-09-17

| MemberID | TypeName            |
|----------|---------------------|
| 12       | Report1789668992879 |

*Figure 4.5. Preferred sport saved for the same member.*

### 4.3.3 Facility Search

Members can combine sport, location, date and time filters. Empty filters are ignored. Both times are required when a time filter is used, and an end time must follow the start time. A query excludes any facility with an overlapping non-cancelled booking. The page shows no-results feedback when no venue matches.

```
if (!ModelState.IsValid) return View(model);

var facilities = _context.Facilities.Include(f => f.Type).AsQueryable();
if (model.TypeId.HasValue) facilities = facilities.Where(f => f.TypeId ==
model.TypeId);
if (!string.IsNullOrEmpty(model.Location))
    facilities = facilities.Where(f => f.Location != null &&
f.Location.Contains(model.Location.Trim()));
if (model.Date.HasValue)
{
    var start = model.Date.Value.ToDateTime(model.StartTime ?? TimeOnly.MinValue);
    var end = model.EndTime.HasValue ?
model.Date.Value.ToDateTime(model.EndTime.Value) : start.AddDays(1);
    facilities = facilities.Where(f => !f.Bookings.Any(b => b.Status != "Cancelled" &&
b.BookingDate == model.Date
&& (b.StartTime == null || b.EndTime == null || (b.StartTime < end &&
b.EndTime > start))));
}
model.Facilities = await facilities.OrderBy(f => f.Name).ToListAsync();
```

SportsBookingSystem   Home   Search Facilities   My Bookings   Hello, Report Evidence Member   Logout   Reviews   Contact Us

## Find a facility

Choose a date and time to find available facilities. With a date only, results show facilities with no bookings that day.

| Sport                              | Location            | Date             | Start time | End time             |
|------------------------------------|---------------------|------------------|------------|----------------------|
| All sports ▾                       | Report Test Area    | 10/17/2026 📅     | 10:00 AM ⌚ | 11:00 AM ⌚           |
| <div>Search</div> <div>Clear</div> |                     |                  |            |                      |
| Facility                           | Sport               | Location         | Capacity   | Hourly rate (LKR)    |
| Report Tennis Court                | Report1789668992879 | Report Test Area | 4          | 1,500.00             |
|                                    |                     |                  |            | <a href="#">Book</a> |

© 2026 - SportsBookingSystem - [Privacy](#)

Figure 4.6. Member search with availability, price and booking link.

## Find a facility

Choose a date and time to find available facilities. With a date only, results show facilities with no bookings that day.

|                                              |                      |            |            |          |
|----------------------------------------------|----------------------|------------|------------|----------|
| Sport                                        | Location             | Date       | Start time | End time |
| Report1789668992879 ▾                        | <input type="text"/> | 10/17/2026 | 10:30 AM   | 11:30 AM |
| <div><div>Search</div><div>Clear</div></div> |                      |            |            |          |

No facilities match your search.

*Figure 4.7. The same test facility is excluded when its slot is booked.*

### 4.3.4 Facility Booking

The form accepts a facility, date, start time and end time. MemberID comes from the session and cannot be chosen through the form. The controller rejects past times, invalid intervals, nonexistent facilities and clashes. New requests are saved as Pending. The My Bookings query filters by the logged-in member, so it does not list another member's bookings.

## Book a facility

Choose a future time slot. Your request will be saved as Pending.

Facility  
Report Tennis Court

Date  
10/17/2026

Start time  
10:00 AM

End time  
11:00 AM

Request booking

Figure 4.8. Booking form before submission.

```
await using var transaction = await _context.Database
    .BeginTransactionAsync(IsolationLevel.Serializable);
var clash = await _context.Bookings.AnyAsync(b =>
    b.FacilityId == model.FacilityId && b.BookingDate == model.BookingDate
    && b.Status != "Cancelled"
    && (b.StartTime == null || b.EndTime == null
        || (b.StartTime < end && b.EndTime > start)));
```

The serializable transaction keeps the availability check and insert together. The overlap rule uses  $\text{start} < \text{requested end}$  and  $\text{end} > \text{requested start}$ . Therefore 10:00–11:00 and 11:00–12:00 do not overlap. Load testing and deadlock retry handling are not included in the current prototype.

Your booking request has been saved.

## My bookings

[Book a facility](#)

| Facility            | Date        | Start | End   | Status  |
|---------------------|-------------|-------|-------|---------|
| Report Tennis Court | 17 Oct 2026 | 10:00 | 11:00 | Pending |

Figure 4.9. Saved booking displayed in My Bookings.

### Saved booking data

SportsBooking - SQL Server result captured 2026-09-17

| BookingID | MemberID | FacilityID | BookingDate | StartTime           | EndTime             | Status    |
|-----------|----------|------------|-------------|---------------------|---------------------|-----------|
| 24        | 12       | 22         | 2026-10-17  | 2026-10-17T10:00:00 | 2026-10-17T11:00:00 | Pending   |
| 25        | 12       | 22         | 2020-01-01  | 2020-01-01T10:00:00 | 2020-01-01T11:00:00 | Confirmed |

Figure 4.10. Saved booking rows used for screenshot evidence. The 2020 Confirmed row was deliberately seeded to test reviews.

### 4.3.5 Review Submission

A member can review only a facility with a Confirmed booking that has already ended. The form provides eligible facilities, an integer rating from 1 to 5 and optional comments up to 255 characters. The server supplies MemberID and ReviewDate. The database also enforces the rating range through CK\_Review\_Rating.

```
if (!await _context.Bookings.AnyAsync(b =>
    b.MemberId == memberId && b.FacilityId == model.FacilityId
    && b.Status == "Confirmed" && b.EndTime < DateTime.Now))
{
    ModelState.AddModelError("FacilityId",
        "You can review a facility after a confirmed booking has ended.");
}
```

SportsBookingSystem   Home   Search Facilities   My Bookings   Hello, Report Evidence Member   Logout   Reviews   Contact Us

## Write a review

You can review facilities after a confirmed booking has ended. If none appear, you have no eligible bookings yet.

Facility

Report Tennis Court

Rating

4

Comments

The court was clean and the booking process was easy.

Submit review

© 2026 - SportsBookingSystem - [Privacy](#)

*Figure 4.11. Review form for an eligible facility.*

## Saved review data

SportsBooking - SQL Server result captured 2026-09-17

| ReviewID | MemberID | FacilityID | Rating | Comments                                              | ReviewDate              |
|----------|----------|------------|--------|-------------------------------------------------------|-------------------------|
| 24       | 12       | 22         | 4      | The court was clean and the booking process was easy. | 2026-09-17T23:46:42.253 |

*Figure 4.12. Saved review with its integer rating and date.*

## 4.4 Guest Functionality

### 4.4.1 Restricted Search

Guests can use the search filters to see facility names, sport types, locations and broad availability. They do not see member-only booking links, capacity or hourly-rate columns. This display rule is backed by server checks on booking and review creation. Direct requests to those actions redirect guests to login.

## Find a facility

Choose a date and time to find available facilities. With a date only, results show facilities with no bookings that day.

Guests can browse availability. [Sign in](#) to see rates and book.

|                                                                            |                      |            |            |          |
|----------------------------------------------------------------------------|----------------------|------------|------------|----------|
| Sport                                                                      | Location             | Date       | Start time | End time |
| All sports ▾                                                               | <input type="text"/> | mm/dd/yyyy | --:-- --   | --:-- -- |
| <input type="button" value="Search"/> <input type="button" value="Clear"/> |                      |            |            |          |

| Facility                | Sport               | Location                          |
|-------------------------|---------------------|-----------------------------------|
| Badminton Hall A        | Badminton           | Town Hall Sports Centre, Colombo  |
| Badminton Hall A        | Badminton           | Town Hall Sports Centre, Colombo  |
| Badminton Hall B        | Badminton           | Town Hall Sports Centre, Colombo  |
| Badminton Hall B        | Badminton           | Town Hall Sports Centre, Colombo  |
| Central Tennis Court 1  | Tennis              | Community Sports Complex, Negombo |
| Central Tennis Court 1  | Tennis              | Community Sports Complex, Negombo |
| Central Tennis Court 2  | Tennis              | Community Sports Complex, Negombo |
| Central Tennis Court 2  | Tennis              | Community Sports Complex, Negombo |
| Indoor Basketball Court | Basketball          | Community Sports Complex, Negombo |
| Indoor Basketball Court | Basketball          | Community Sports Complex, Negombo |
| Municipal Soccer Ground | Soccer              | Municipal Grounds, Colombo        |
| Municipal Soccer Ground | Soccer              | Municipal Grounds, Colombo        |
| Municipal Swimming Pool | Swimming            | Municipal Grounds, Colombo        |
| Municipal Swimming Pool | Swimming            | Municipal Grounds, Colombo        |
| Open Basketball Court   | Basketball          | Riverside Park, Gampaha           |
| Open Basketball Court   | Basketball          | Riverside Park, Gampaha           |
| Report Tennis Court     | Report1789668992879 | Report Test Area                  |
| Riverside Soccer Field  | Soccer              | Riverside Park, Gampaha           |
| Riverside Soccer Field  | Soccer              | Riverside Park, Gampaha           |

Figure 4.13. Guest search without member-only booking controls.

```
var memberId = HttpContext.Session.GetInt32("MemberID");
if (memberId == null)
    return RedirectToAction("Login", "Account");
```

Unused scaffolded Member, FacilityType and SportPreference controllers are marked NonController. Facility, Booking, Review and Inquiry controllers expose only the intended routes. Old management URLs return 404; hiding links alone would not protect the underlying actions.

### 4.4.2 Review Search

Guests can read reviews without an account. Optional filters select a facility and/or rating. The query includes Facility and Member so the page shows meaningful names instead of foreign-key numbers. The public page has no edit or delete links.

## Facility reviews

|                     |        |                        |                       |
|---------------------|--------|------------------------|-----------------------|
| Facility            | Rating |                        |                       |
| Report Tennis Court | 4 / 5  | <a href="#">Search</a> | <a href="#">Clear</a> |

### Report Tennis Court — 4 / 5

The court was clean and the booking process was easy.

Report Evidence Member · 17 Sep 2026

Figure 4.14. A guest reads the submitted four-star review.

```
var reviews = _context.Reviews
    .Include(r => r.Facility).Include(r => r.Member).AsQueryable();
if (facilityId.HasValue)
    reviews = reviews.Where(r => r.FacilityId == facilityId);
if (rating.HasValue)
    reviews = reviews.Where(r => r.Rating == rating.Value);
```

### 4.4.3 Membership Registration

The home page and guest navigation both link to Account/Register. Guests use the same registration form described in Section 4.3.2, which includes the required name and email as well as contact details and sport preferences. After a successful submission the guest becomes a member and reaches the member search page.

### 4.4.4 Send Inquiry

Inquiry/Create is public. Name, a valid email and message are required. The application sets DateSent, saves the row and displays a confirmation. Inquiry does not need a MemberID. The application no longer exposes a public list of other people's inquiries.

## Contact the Sports Council

Your name

Report Evidence Guest

Email

Report1789668992879@example.test

Message

Are tennis coaching sessions available on weekends?

Send inquiry

*Figure 4.15. Guest inquiry form.*

```
if (!ModelState.IsValid) return View(inquiry);
inquiry.DateSent = DateOnly.FromDateTime(DateTime.Today);
_context.Inquiries.Add(inquiry);
await _context.SaveChangesAsync();
TempData["Success"] = "Your inquiry has been sent to the Sports Council.";
```

Your inquiry has been sent to the Sports Council.

## Contact the Sports Council

Your name

Email

Message

Send inquiry

Figure 4.16. Confirmation after saving an inquiry.

### Saved inquiry data

SportsBooking - SQL Server result captured 2026-09-17

| InquiryID | GuestName             | Message                                             | DateSent   |
|-----------|-----------------------|-----------------------------------------------------|------------|
| 12        | Report Evidence Guest | Are tennis coaching sessions available on weekends? | 2026-09-17 |

Figure 4.17. Corresponding Inquiry row in SQL Server.

## 5. Testing

Testing used the running application and local SQL Server database on 17 September 2026. The PowerShell script tests HTTP routes, validation and persistence. A separate headless Microsoft Edge run checks rendered pages and captures screenshots. Both use uniquely identified temporary records and remove them afterwards. The screenshots show deliberate test data, including a past Confirmed booking needed for review eligibility.

| ID  | Test and data                                   | Expected result                        | Actual result                        | Status |
|-----|-------------------------------------------------|----------------------------------------|--------------------------------------|--------|
| T01 | Home page<br>GET /                              | Sports information visible             | Home heading returned                | Pass   |
| T02 | Guest facility search<br>Type filter; no login  | Browse without rate or booking details | Facility visible; rate column absent | Pass   |
| T03 | Guest booking form<br>GET /Booking/Create       | Redirect to login                      | Login redirect                       | Pass   |
| T04 | Guest booking list<br>GET /Booking/MyBookings   | Redirect to login                      | Login redirect                       | Pass   |
| T05 | Guest review form<br>GET /Review/Create         | Redirect to login                      | Login redirect                       | Pass   |
| T06 | Member management<br>GET /Member                | Route disabled                         | 404                                  | Pass   |
| T07 | Facility creation<br>GET /Facility/Create       | Route disabled                         | 404                                  | Pass   |
| T08 | Facility editing<br>GET /Facility/Edit/1        | Route disabled                         | 404                                  | Pass   |
| T09 | Review deletion<br>GET /Review/Delete/1         | Route disabled                         | 404                                  | Pass   |
| T10 | Inquiry list<br>GET /Inquiry                    | Route disabled                         | 404                                  | Pass   |
| T11 | Preference management<br>GET /SportPreference   | Route disabled                         | 404                                  | Pass   |
| T12 | Sport management<br>GET /FacilityType           | Route disabled                         | 404                                  | Pass   |
| T13 | Invalid login<br>Unknown email, wrong password  | Display error                          | Invalid credentials message          | Pass   |
| T14 | Invalid sport<br>SelectedSports=999999          | Reject unknown sport                   | Validation message                   | Pass   |
| T15 | Registration<br>Valid new details and sport     | Save and sign in                       | Redirect to search                   | Pass   |
| T16 | Preference insert<br>New member and sport IDs   | One matching row                       | SQL count = 1                        | Pass   |
| T17 | Duplicate email<br>Register same email again    | Reject duplicate                       | Already registered message           | Pass   |
| T18 | Valid login<br>New test email, correct password | Sign in                                | Redirect to search                   | Pass   |

| ID  | Test and data                                                 | Expected result               | Actual result              | Status |
|-----|---------------------------------------------------------------|-------------------------------|----------------------------|--------|
| T19 | Member search<br>Logged-in type search                        | Show rate and Book link       | Both shown                 | Pass   |
| T20 | Booking save<br>Future date, 10:00–11:00                      | Save booking                  | Confirmation shown         | Pass   |
| T21 | Forged booking fields<br>MemberId=999999,<br>Status=Confirmed | Use session ID and<br>Pending | SQL verified server values | Pass   |
| T22 | Overlap<br>Existing 10:00–11:00; request 10:30–<br>11:30      | Reject clash                  | Already booked message     | Pass   |
| T23 | Availability search<br>Search booked 10:30–11:30              | Exclude booked facility       | No results                 | Pass   |
| T24 | Adjacent slot<br>Request 11:00–12:00                          | Allow booking                 | Confirmation shown         | Pass   |
| T25 | Invalid interval<br>Start 12:00, end 11:00                    | Reject interval               | End-time validation        | Pass   |
| T26 | Past booking<br>1 January 2020                                | Reject past time              | Future-time validation     | Pass   |
| T27 | Review before visit<br>Only future Pending booking            | Reject review                 | Eligibility validation     | Pass   |
| T28 | Invalid rating<br>Eligible visit, rating 6                    | Reject out-of-range value     | Range validation           | Pass   |
| T29 | Valid review<br>Eligible visit, rating 4                      | Save review                   | Confirmation shown         | Pass   |
| T30 | Review filter<br>Facility and rating=4                        | Matching review visible       | Expected content returned  | Pass   |
| T31 | Empty inquiry<br>Empty Message                                | Reject required field         | Required-field validation  | Pass   |
| T32 | Inquiry submit<br>Valid name, email and message               | Save and confirm              | Confirmation shown         | Pass   |
| T33 | Inquiry persistence<br>Query saved inquiry                    | One row with today's date     | SQL count = 1              | Pass   |
| T34 | Logout<br>POST /Account/Logout                                | Clear session                 | Protected page redirects   | Pass   |

## 5.1 Test Evidence

Figures 4.2–4.17 support the login, registration, booking, review and inquiry tests. The image below records the overlap validation from T22. The saved test script contains the repeatable requests and SQL assertions. The browser run separately confirms guest review access using a fresh, unauthenticated browser context.

## Book a facility

Choose a future time slot. Your request will be saved as Pending.

- This facility is already booked during that time.

Facility

Report Tennis Court

Date

10/17/2026

Start time

10:30 AM

End time

11:30 AM

Request booking

© 2026 - SportsBookingSystem - [Privacy](#)

Figure 5.1. T22: an overlapping booking produces an error.

| Browser check                      | Result |
|------------------------------------|--------|
| Guest redirected from booking form | Pass   |
| Invalid login rejected             | Pass   |
| Registration opens member search   | Pass   |
| Booking confirmation displayed     | Pass   |
| Overlapping booking rejected       | Pass   |
| Booked slot absent from search     | Pass   |
| Review saved                       | Pass   |
| Guest reads saved review           | Pass   |
| Inquiry confirmation displayed     | Pass   |

## 5.2 Summary of Testing

All 34 automated HTTP/database checks passed. All nine additional browser checks passed. The project built with zero warnings and zero errors. These results cover the tested prototype flows; they do not establish production security, performance under load or complete browser compatibility.

| Fault found                                            | Correction                                          | Retest               |
|--------------------------------------------------------|-----------------------------------------------------|----------------------|
| Search and My Bookings links targeted missing actions. | Implemented Facility/Search and Booking/MyBookings. | T15, T18–T20 passed. |

| Fault found                                                        | Correction                                                             | Retest                                                        |
|--------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------|
| Scaffolded booking allowed posted identity/status fields.          | Used a form view model; set identity and Pending status on the server. | T21 passed.                                                   |
| Booking ID mapping did not match SQL identity columns.             | BookingID generated on add; MemberID supplied normally.                | T20 inserted a booking successfully.                          |
| Review rating and report disagreed; CHECK constraints were absent. | Converted valid ratings to INT, updated EF and added both checks.      | T28–T29 passed; database metadata verified.                   |
| CSS did not load when running without a launch profile.            | Added UseStaticFiles before routing.                                   | Bootstrap resource returned HTTP 200; screenshots recaptured. |
| Unused scaffolded routes exposed management operations.            | Disabled unused controllers and removed public CRUD actions.           | T06–T12 returned 404.                                         |

Remaining limits include plain-text passwords, no automated booking approval, repeated reviews, no pagination and no concurrent-load test. The booking transaction uses serializable isolation, but a production version should handle deadlocks and retries. Form checks reject invalid user inputs; further database checks for times, rates and capacity would strengthen direct SQL writes.

## 6. Further Discussion and Reflection

The main design decision in this project was to keep SQL Server as the source of the data model. This made it possible to compare the database diagram, SQL script, entity classes and report directly. It also exposed a weakness in relying on generated files without checking them: the Booking key mapping and Review rating type did not match the intended design.

For this prototype, I kept the controllers and Razor pages simple. Each form has a small view model and the controller handles its validation and database operation. This makes it easier to follow the booking flow from the submitted date and time to the final Booking row. A larger application would need more separation of business rules and reusable authorization checks.

The booking overlap rule was an important part of the implementation. Comparing only the start times would miss partially overlapping reservations. The implemented rule compares both ends of each interval. Tests confirmed that an overlapping request is rejected while an adjacent booking is accepted. The server also supplies MemberID and Status, which prevents a member from changing them by editing form data.

The review flow links feedback to a completed Confirmed booking. This makes the meaning of a review clearer, but it also exposes a limitation in the workflow: the prototype has no approval screen. New requests stay Pending until the database is updated manually. An administrative approval page is therefore a useful next feature.

Testing revealed that a successful build does not prove the whole page works. The CSS problem appeared only when the application was opened for screenshots without a launch profile. The application also needed checks that direct URLs were blocked for guests, because removing a navigation link alone does not protect an action.

The project would need several improvements before public use. Passwords should be hashed, session-based identity checks should be replaced or strengthened with standard authentication, booking approvals should be managed through a protected interface, and confirmations could be sent by email. Search paging and broader concurrency tests would help with larger data sets. The practical lesson from the verified changes is to keep the schema, code, screenshots and written claims consistent, and to support each main feature with a repeatable test.

## 7. Conclusion

The prototype provides member registration, login, facility search, booking requests, reviews and guest inquiries using ASP.NET Core MVC and SQL Server. The seven-table schema supports the required relationships, and the rating and status rules are enforced in the database. The tested application passed 34 HTTP/database checks and nine browser checks. It remains a coursework prototype with the limitations discussed above.

## 8. References

Microsoft (n.d.-a). Overview of ASP.NET Core MVC. Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0> (Accessed: 17 September 2026).

Microsoft (n.d.-b). Reverse Engineering. Entity Framework Core. Available at: <https://learn.microsoft.com/en-us/ef/core/managing-schemas/scaffolding/> (Accessed: 17 September 2026).

Microsoft (n.d.-c). Session in ASP.NET Core. Available at: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state?view=aspnetcore-10.0> (Accessed: 17 September 2026).

Microsoft (n.d.-d). Model validation in ASP.NET Core MVC. Available at: <https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-10.0> (Accessed: 17 September 2026).

Oracle (n.d.). Data Modeler Concepts and Usage. SQL Developer Data Modeler 24.3 User's Guide. Available at: <https://docs.oracle.com/en/database/oracle/sql-developer-data-modeler/24.3/dmdug/data-modeler-concepts-usage.html> (Accessed: 17 September 2026).

Coursework brief (undated). Community Sports Facilities Booking System. Supplied assignment document.

Project evidence: SportsBookingSystem source files, ER.dmd, ERD.png, Logical.png, ERDB File.ddl, SQL metadata and query results, SmokeTest.ps1 output and browser screenshots captured on 17 September 2026.

## 9. Appendices

### Appendix A Full database schema

This script recreates the verified schema in an empty database. It is a schema export, not an instruction to drop or recreate the existing SportsBooking database.

#### Member

```
CREATE TABLE Member (  
    MemberID INT IDENTITY(1,1) NOT NULL,  
    Name VARCHAR(255) NOT NULL,  
    Email VARCHAR(255) NOT NULL,  
    Phone VARCHAR(255) NULL,  
    Address VARCHAR(255) NULL,  
    Password VARCHAR(255) NOT NULL,  
    RegDate DATE NULL,  
    CONSTRAINT Member__UN UNIQUE (Email),  
    CONSTRAINT Member_PK PRIMARY KEY (MemberID)  
);  
GO
```

#### FacilityType

```
CREATE TABLE FacilityType (  
    TypeID INT IDENTITY(1,1) NOT NULL,  
    TypeName VARCHAR(255) NOT NULL,  
    CONSTRAINT FacilityType_PK PRIMARY KEY (TypeID)  
);  
GO
```

#### Facility

```
CREATE TABLE Facility (  
    FacilityID INT IDENTITY(1,1) NOT NULL,  
    Name VARCHAR(255) NOT NULL,  
    TypeID INT NOT NULL,  
    Location VARCHAR(255) NULL,  
    Capacity INT NULL,  
    HourlyRate DECIMAL(25,2) NULL,  
    CONSTRAINT Facility_PK PRIMARY KEY (FacilityID),  
    CONSTRAINT Facility_FacilityType_FK FOREIGN KEY (TypeID) REFERENCES FacilityType(TypeID)  
);  
GO
```

#### Booking

```
CREATE TABLE Booking (  
    BookingID INT IDENTITY(1,1) NOT NULL,  
    MemberID INT NOT NULL,  
    FacilityID INT NOT NULL,  
    BookingDate DATE NOT NULL,  
    StartTime DATETIME NULL,  
    EndTime DATETIME NULL,  
    Status VARCHAR(255) NOT NULL,  
    CONSTRAINT Booking_PK PRIMARY KEY (BookingID),  
    CONSTRAINT Booking_Facility_FK FOREIGN KEY (FacilityID) REFERENCES Facility(FacilityID),  
    CONSTRAINT Booking_Member_FK FOREIGN KEY (MemberID) REFERENCES Member(MemberID),  
    CONSTRAINT CK_Booking_Status CHECK ([Status]='Cancelled' OR [Status]='Confirmed' OR  
[Status]='Pending')  
);  
GO
```

## Review

```
CREATE TABLE Review (
    ReviewID INT IDENTITY(1,1) NOT NULL,
    MemberID INT NOT NULL,
    FacilityID INT NOT NULL,
    Rating INT NOT NULL,
    Comments VARCHAR(255) NULL,
    ReviewDate DATETIME NULL,
    CONSTRAINT Review_PK PRIMARY KEY (ReviewID),
    CONSTRAINT Review_Facility_FK FOREIGN KEY (FacilityID) REFERENCES Facility(FacilityID),
    CONSTRAINT Review_Member_FK FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    CONSTRAINT CK_Review_Rating CHECK ([Rating]>=(1) AND [Rating]<=(5))
);
GO
```

## SportPreference

```
CREATE TABLE SportPreference (
    SportPreferenceID INT IDENTITY(1,1) NOT NULL,
    MemberID INT NOT NULL,
    TypeID INT NOT NULL,
    CONSTRAINT SportPreference__UN UNIQUE (MemberID, TypeID),
    CONSTRAINT SportPreference_PK PRIMARY KEY (SportPreferenceID),
    CONSTRAINT SportPreference_FacilityType_FK FOREIGN KEY (TypeID) REFERENCES
FacilityType(TypeID),
    CONSTRAINT SportPreference_Member_FK FOREIGN KEY (MemberID) REFERENCES Member(MemberID)
);
GO
```

## Inquiry

```
CREATE TABLE Inquiry (
    InquiryID INT IDENTITY(1,1) NOT NULL,
    GuestName VARCHAR(255) NULL,
    Email VARCHAR(255) NULL,
    Message VARCHAR(255) NULL,
    DateSent DATE NULL,
    CONSTRAINT Inquiry_PK PRIMARY KEY (InquiryID)
);
GO
```

## Appendix B Sample insert script

This example inserts one related set of sample records in a test database. The existing database's wider sample data and actual row counts are documented in Section 3.2.

```
SET XACT_ABORT ON;
BEGIN TRANSACTION;
INSERT INTO Member (Name,Email,Phone,Address>Password,RegDate)
VALUES ('Sample Member','coursework.sample@example.test','0770000000',
'Negombo','Sample123!',CAST(GETDATE() AS DATE));
DECLARE @MemberID INT = SCOPE_IDENTITY();
INSERT INTO FacilityType (TypeName) VALUES ('Sample Tennis');
DECLARE @TypeID INT = SCOPE_IDENTITY();
INSERT INTO Facility (Name,TypeID,Location,Capacity,HourlyRate)
VALUES ('Sample Tennis Court',@TypeID,'Negombo',4,1500.00);
DECLARE @FacilityID INT = SCOPE_IDENTITY();
INSERT INTO SportPreference (MemberID,TypeID) VALUES (@MemberID,@TypeID);
INSERT INTO Booking (MemberID,FacilityID,BookingDate,StartTime,EndTime,Status)
VALUES (@MemberID,@FacilityID,'2026-08-01','2026-08-01T09:00:00',
'2026-08-01T10:00:00','Confirmed');
INSERT INTO Review (MemberID,FacilityID,Rating,Comments,ReviewDate)
VALUES (@MemberID,@FacilityID,4,'The court was clean.','2026-08-01T11:00:00');
INSERT INTO Inquiry (GuestName,Email,Message,DateSent)
VALUES ('Sample Guest','guest@example.test','Are weekend sessions available?',
CAST(GETDATE() AS DATE));
COMMIT;
```

## Appendix C Running and verifying the project

Open SportsBookingSystem.csproj in Visual Studio or use the commands below. The configured database is SportsBooking on 127.0.0.1 with Windows authentication. SQL Server must be running, and the Windows account must have access.

```
dotnet build
dotnet run --no-launch-profile --urls http://127.0.0.1:5187
# Run in a second PowerShell window.
.\tests\SmokeTest.ps1
```

The test script creates and removes only its uniquely named temporary member, facility, category, bookings, reviews and inquiry records. Its SQL cleanup runs in finally. It should be run against a local coursework/test database.

## Appendix D Full controller code

The complete controller source is supplied with the project. The main report includes the relevant excerpts to keep the explanations readable. Files: Controllers/AccountController.cs, FacilityController.cs, BookingController.cs, ReviewController.cs and InquiryController.cs. Program.cs contains service and middleware setup.